

PROJET ETSIA

« YANSNET – PARTIE APPLICATION »

BILAN DE PROJET

GROUPE PROJET 15

MAKONGNE DEFFO July Merveille
NANKAP NDIZEU Loic Aurel
NKOULOU Joseph Emmanuel
NOUNGOUA NOUNGOUA Teddy Steven
TOFA DEFFO Lionel Junior

Promotion X2027

ANNEE ACADEMIQUE 2024 – 2025

Sommaire

1. Présentation du projet.....	3
2. Objectifs atteints et non atteints	4
2.1 Tableau de synthèse	4
2.2 Objectifs atteints.....	4
2.3 Objectifs partiellement atteints.....	5
2.4 Objectifs non atteints	5
3. Difficultés rencontrées.....	6
3.1 Difficultés techniques	6
3.2 Difficultés organisationnelles	7
4. Retours d'expérience techniques	8
4.1 Architecture et choix technologiques	8
4.2 Intégration de l'Intelligence Artificielle	9
4.3 Sécurité et conformité des données.....	9
5. Retours d'expérience organisationnels.....	10
5.1 Travail en équipe et coordination	10
5.2 Gestion du temps et priorisation	10
6. Recommandations pour la suite.....	11
7. Conclusion.....	12

1. Présentation du projet

YANSNET est un réseau social académique développé pour les étudiants et le personnel de l'UCAC-ICAM de Douala. Il offre un espace numérique sécurisé de communication, de partage de contenu et de veille psychologique, conçu pour fonctionner sur le réseau local de la cité Yansoki.

Le projet a été réalisé dans le cadre du cours ETSIA (Études et Techniques des Systèmes d'Information Avancés), Promotion X2027, par le Groupe 15. Le sprint de réalisation a duré 6 semaines et a mobilisé cinq étudiants répartis sur trois composantes techniques :

- Application mobile : React Native / Expo (iOS & Android)
- Interface administrateur web : React 18 / Vite / TypeScript
- Backend API : Node.js / Express (port 3000)
- Serveur Data & IA : Python / Flask (port 5002) + modèles HuggingFace
- Base de données : PostgreSQL 12+ (27 tables, 8 domaines fonctionnels)

Ce bilan documente honnêtement ce qui a été accompli, ce qui n'a pas pu l'être, les difficultés rencontrées et les enseignements tirés de cette expérience de développement en équipe.

2. Objectifs atteints et non atteints

2.1 Tableau de synthèse

Le tableau ci-dessous dresse le bilan de l'ensemble des objectifs fonctionnels prévus au cahier des charges :

Objectif	Statut	Commentaire
Authentification sécurisée (JWT, email institutionnel)	ATTEINT	Login, register, refresh token fonctionnels
Publication de posts avec modération IA	ATTEINT	Harcèlement + NSFW détectés avant insertion

Messagerie privée chiffrée E2E	ATTEINT	Fernet AES-128, WebSocket temps réel
Groupes de discussion multi-utilisateurs	ATTEINT	Création, adhésion, messages temps réel
Système de likes, commentaires, follows	ATTEINT	Interactions sociales complètes avec notifications
Analyse sentimentale & alertes SOS concierges	ATTEINT	Batch 6h, seuil critique 0.85, alerte automatique
Interface administrateur web (React/Vite)	ATTEINT	Dashboard, gestion users, file de modération
Recommandations personnalisées (posts & amis)	PARTIEL	Algorithme basique livré, collaborative filtering reporté
Notifications push Firebase mobile	NON ATTEINT	Reporté — hors périmètre MVP du sprint
Analytics avancés pour concierges	NON ATTEINT	Vues basiques disponibles, tableaux de bord complets reportés

Légende : **ATTEINT** = livré et fonctionnel | **PARTIEL** = livré avec périmètre réduit | **NON ATTEINT** = reporté hors sprint

2.2 Objectifs atteints

Les fonctionnalités suivantes ont été entièrement développées, testées et intégrées dans la version livrée :

Système d'authentification sécurisé

L'inscription avec validation de l'email institutionnel (@ucac-icam.com), la connexion avec hachage bcrypt, la génération de tokens JWT (7 jours) et le refresh token sans re-saisie du mot de passe sont entièrement fonctionnels. Le contrôle d'accès basé sur les rôles (RBAC, 5 rôles) est appliqué sur toutes les routes protégées.

Publication de contenu avec modération IA intégrée

Chaque post passe par une chaîne de modération synchrone avant publication : détection de harcèlement (toxic-bert, seuil 0.70) et détection NSFW des images (Falconsai/nsfw_image_detection). Les contenus rejetés sont notifiés à l'auteur avec la raison du refus. La file de modération manuelle capture les cas ambigus.

Messagerie privée chiffrée et discussions de groupe

Les messages privés sont chiffrés end-to-end via Fernet (AES-128) avant stockage — même les administrateurs ne peuvent pas les lire en clair. La messagerie de groupe (non chiffrée, visible par tous les membres) est distribuée en temps réel via Socket.io.

Interactions sociales complètes

Likes (idempotents, avec anti-doublon), commentaires (passant par l'IA avant insertion), follows/unfollows et notifications en temps réel sont entièrement implémentés. Le rate limiting par action protège les serveurs contre le spam.

Analyse sentimentale & alertes SOS

Le modèle distilbert-base-multilingual analyse les posts de chaque utilisateur en batch toutes les 6 heures. Si le score de négativité dépasse 0.85 sur 7 jours, une alerte SOS est envoyée aux concierges. Ce mécanisme constitue l'une des fonctionnalités les plus innovantes du projet.

Interface administrateur web

Le tableau de bord administrateur (React/Vite/TypeScript) permet la gestion des utilisateurs, la modération manuelle du contenu, la consultation des logs d'audit et la supervision des alertes SOS. L'authentification est partagée avec le Backend via le même JWT.

2.3 Objectifs partiellement atteints

Recommandations personnalisées

Un système de recommandations basique a été livré : recommandation de posts trending et d'amis partageant la même filière. En revanche, le collaborative filtering avancé basé sur content_engagement (prévu pour personnaliser finement les suggestions selon les interactions passées) n'a pas pu être développé dans le sprint.

L'endpoint GET /recommandations/:userId est fonctionnel mais retourne des résultats moins personnalisés qu'initialement prévu.

2.4 Objectifs non atteints

Notifications push Firebase

L'intégration de Firebase Cloud Messaging pour les notifications push sur les appareils mobiles a été identifiée comme hors périmètre MVP dès la deuxième semaine du sprint. Les notifications temps réel via Socket.io (in-app) ont été privilégiées. Firebase sera intégré dans une itération future.

Analytics avancés pour concierges

Des vues d'activité basiques sont disponibles dans l'interface admin. Les tableaux de bord concierges complets (graphiques d'évolution du bien-être, comparaisons par filière et promotion, exports CSV) n'ont pas été développés. Ce module nécessitait une semaine supplémentaire non disponible dans le sprint.

3. Difficultés rencontrées

3.1 Difficultés techniques

Téléchargement et déploiement des modèles IA (~2 Go)

Le premier lancement du serveur Data nécessite le téléchargement d'environ 500 Mo de modèles HuggingFace. Sur le réseau du LAN Yansoki, ce téléchargement a échoué à plusieurs reprises à cause de timeouts SSL et d'un proxy réseau non configuré. La solution a été de pré-télécharger les modèles en dehors du campus et de les distribuer dans le dossier Code/app/data_models/ sous contrôle de version Git LFS. Les fichiers de modèles représentent au total ~2 Go, ce qui a également posé des problèmes lors du push vers GitHub (limite de 100 Mo par fichier).

Environnement Python et gestion des dépendances

Plusieurs membres de l'équipe ont rencontré des corruptions d'environnements virtuels Python après des interruptions d'installation (coupures de courant, fermeture forcée du terminal). pip devenait inaccessible dans le venv, bloquant

l'installation de torch, transformers et safetensors. La solution systématique a été de supprimer et recréer le venv depuis zéro, puis de réinstaller via `pip install -r requirements_data.txt`.

Import PostgreSQL et configuration de la base de données

L'import du backup SQL via l'interface graphique pgAdmin a échoué car le Query Tool ne supporte pas la commande COPY avec des chemins absolus locaux. Tous les membres n'étaient pas familiers avec la ligne de commande psql. Une documentation d'import (`psql -U postgres -d yansnet_db -f backup.sql`) a dû être rédigée et diffusée à l'équipe.

Intégration WebSocket entre backend et clients

La synchronisation temps réel via Socket.io a nécessité plusieurs itérations pour gérer correctement la reconnexion automatique des clients mobiles (Expo) et la gestion des rooms Socket.io pour les groupes. Les événements émis sans listener actif (utilisateur hors ligne) devaient être mis en file d'attente et distribués à la reconnexion.

Compatibilité React Native / Expo sur Android et iOS

Certaines librairies npm (notamment celles nécessitant des modules natifs compilés) étaient incompatibles avec le SDK Expo géré. L'utilisation d'Expo Go a limité l'accès aux fonctionnalités natives avancées (notifications push, accès à certains capteurs). Le projet a dû rester dans le périmètre des API Expo disponibles.

3.2 Difficultés organisationnelles

Coordination entre équipes Backend et Data

Les équipes Node.js et Flask ont développé en parallèle pendant les deux premières semaines sans contrat d'interface formel. Des désalignements sont apparus sur les noms de champs JSON (texte vs text, user_id vs userId) et sur les codes HTTP retournés (200 vs 201 pour les créations). La résolution a nécessité une réunion de synchronisation d'urgence en semaine 3 et la mise en place d'un fichier de contrats API partagé sur Git.

Sous-estimation de la complexité du sprint

Le volume de fonctionnalités prévu au cahier des charges représentait un travail d'environ 10 à 12 semaines pour une équipe de 5 personnes. Le sprint de 6 semaines a contraint l'équipe à prioriser strictement et à renoncer à Firebase et aux analytics avancés. Cette sous-estimation initiale de la charge de travail est le principal enseignement organisationnel du projet.

Gestion des conflits Git en développement parallèle

Le travail simultané sur les mêmes fichiers de configuration (package.json, .env, schéma SQL) a généré des conflits Git récurrents. L'absence d'une stratégie de branches formelle (feature branches) en début de projet a ralenti le cycle d'intégration. À partir de la semaine 4, l'équipe a adopté une convention simple : une branche par fonctionnalité, merge dans main uniquement après validation locale.

Disponibilité inégale des membres de l'équipe

Les contraintes de cours parallèles (TD, TP d'autres modules) ont rendu certaines périodes de la semaine inexploitable pour le projet. Les pics de travail se sont concentrés en soirée et le week-end, ce qui a parfois affecté la qualité du code produit sous pression. Une meilleure planification des créneaux projet dès le début du sprint aurait amélioré la régularité de la production.

4. Retours d'expérience techniques

4.1 Architecture et choix technologiques

L'architecture multi-serveurs (Backend Node.js + Data Flask + Frontend React Native) s'est avérée pertinente pour séparer les responsabilités, mais a introduit une complexité d'intégration sous-estimée. Voici les principaux enseignements :

- L'utilisation de React Native avec Expo a fortement accéléré le développement mobile cross-platform. Une seule base de code a suffi pour cibler Android et iOS, avec des tests facilités par Expo Go.
- Le choix de PostgreSQL a été judicieux pour la complexité relationnelle du schéma (27 tables, clés étrangères, cascades). Une base NoSQL comme MongoDB aurait compliqué les requêtes multi-tables.

- Socket.io a permis une implémentation temps réel robuste mais la gestion des rooms et de la reconnexion a été plus complexe que prévu. Une documentation de la stratégie WebSocket dès le début aurait évité des bugs.
- L'absence d'un contrat API formalisé (OpenAPI/Swagger) entre les équipes Backend et Data dès le jour 1 a causé plusieurs jours de retard en intégration. Un fichier swagger.yaml partagé est fortement recommandé.
- Git LFS pour les gros modèles IA a fonctionné mais le quota de bande passante GitHub LFS a été atteint rapidement. Pour un prochain projet, héberger les modèles sur un service dédié (HuggingFace Hub, S3) serait plus adapté.

4.2 Intégration de l'Intelligence Artificielle

L'intégration des modèles de Machine Learning dans un pipeline de production a été l'aspect le plus formateur et le plus complexe du projet :

- L'utilisation de modèles pré-entraînés HuggingFace (distilbert, toxic-bert, Falconsai) a permis d'atteindre des performances de modération de qualité professionnelle sans expertise en deep learning. Le fine-tuning n'a pas été nécessaire pour les cas d'usage du projet.
- La séparation du serveur IA en service indépendant (Flask, port 5002) a isolé les dépendances Python lourdes du Backend Node.js. En cas de crash du serveur Data, le Backend continue en mode dégradé.
- L'analyse sentimentale en batch (toutes les 6h) est un bon compromis entre pertinence et performance. Un traitement en temps réel aurait saturé les ressources du serveur sur le LAN Yansoki.
- Le seuil de rejet du harcèlement fixé à 0.70 génère quelques faux positifs sur des expressions familières entre étudiants. Un système de feedback utilisateur (contestation d'un rejet) permettrait d'affiner le seuil dynamiquement.

- Les temps de chargement des modèles au démarrage (~30 secondes) impactent le démarrage à froid du serveur Data. Un mécanisme de warm-up et de health check avec retry automatique côté Backend aurait amélioré la résilience.

4.3 Sécurité et conformité des données

La sécurité a été traitée comme une contrainte de conception dès le départ, et non comme un ajout tardif. Les décisions prises et leurs résultats :

- Le chiffrement E2E des messages privés via Fernet garantit la confidentialité même en cas d'accès direct à la base de données. C'est un point fort distinctif de YANSNET par rapport à des solutions similaires.
- Le soft delete (is_active = false) sur toutes les suppressions préserve les données pour l'audit et facilite la conformité RGPD. Aucune donnée n'est physiquement détruite.
- L'audit log complet (user, timestamp, action, paramètres) offre une traçabilité totale des actions administrateur.
- La clé de compte de service Google (serviceAccountKey.json) a été accidentellement incluse dans un commit Git et détectée par GitHub Push Protection. La clé a été révoquée et régénérée. Cet incident souligne l'importance des variables d'environnement et d'un .gitignore rigoureux dès le premier commit.

5. Retours d'expérience organisationnels

5.1 Travail en équipe et coordination

Cette expérience de développement en équipe de 5 personnes sur un projet multi-composantes a mis en évidence des pratiques efficaces et des axes d'amélioration :

- La répartition par composante technique (un référent Backend, un référent Data, deux référents Mobile/Frontend, un référent BD) a permis à chacun de développer une expertise approfondie sur son périmètre.
-

- Le recours à Git comme outil de collaboration central a fonctionné, mais l'adoption tardive des feature branches a causé des conflits évitables. À l'avenir, définir une convention de branches (main/develop/feature/xxx) dès le jour 1.
- Les réunions de synchronisation informelles ont été efficaces mais irrégulières. Un point quotidien court (15 minutes, format stand-up) aurait permis de détecter les blocages plus tôt.
- La documentation interne (API, schéma de BD, instructions d'installation) a été produite en parallèle du code et non en fin de sprint. Cela a facilité l'intégration et l'onboarding des membres qui reprenaient le travail d'un autre.

5.2 Gestion du temps et priorisation

La gestion du temps a été le principal défi organisationnel du projet :

- L'adoption d'un MVP strictement défini dès la semaine 1 a permis de livrer un produit fonctionnel complet sur les fonctionnalités essentielles, plutôt qu'un produit incomplet sur toutes les fonctionnalités.
- L'utilisation de modèles pré-entraînés HuggingFace a économisé plusieurs semaines de développement ML. Cette décision de faire plutôt qu'inventer a été déterminante pour le respect des délais.
- La sous-estimation initiale de la charge de travail (10-12 semaines de travail pour un sprint de 6 semaines) aurait pu être évitée par une estimation plus fine (story points, planning poker) lors du cadrage initial.
- Les fonctionnalités non livrées (Firebase, analytics avancés) ont été documentées et priorisées pour une itération future, ce qui évite de les perdre et facilite la reprise du projet.

6. Recommandations pour la suite

Sur la base des enseignements de ce sprint, l'équipe formule les recommandations suivantes pour les équipes qui reprendraient ou étendraient le projet YANSNET :

Court terme (itération suivante)

- Intégrer Firebase Cloud Messaging pour les notifications push mobiles (iOS et Android) — code de base déjà préparé
- Compléter le collaborative filtering des recommandations en utilisant la table `content_engagement` désormais renseignée
- Développer les tableaux de bord analytics concierges avec graphiques d'évolution du bien-être par filière
- Ajouter un système de feedback utilisateur sur les rejets IA (contestation d'un commentaire rejeté) pour affiner les seuils

Moyen terme (déploiement production)

- Migrer vers une infrastructure cloud (serveur VPS ou hébergement institutionnel) pour sortir de la dépendance au LAN Yansoki
- Mettre en place un pipeline CI/CD (GitHub Actions) pour les tests automatisés et le déploiement continu
- Adopter Docker Compose pour standardiser les environnements de développement et éliminer les problèmes de configuration locale
- Héberger les modèles IA sur HuggingFace Hub ou AWS S3 pour s'affranchir de la limite de 100 Mo de GitHub

Bonnes pratiques à adopter dès le premier jour

- Rédiger le contrat API (Swagger/OpenAPI) avant de coder la moindre route — cela élimine les désalignements entre équipes
- Configurer le `.gitignore` et les secrets (variables d'environnement) avant le premier commit — ne jamais laisser de clés dans le code
- Adopter une convention de branches Git (main/develop/feature/xxx) et un processus de code review minimal
- Planifier des stand-ups quotidiens de 15 minutes pour détecter les blocages avant qu'ils ne deviennent critiques

7. Conclusion

YANSNET est un projet ambitieux qui a mobilisé l'ensemble des compétences acquises au cours de la formation : développement mobile, architecture backend, intégration d'IA, sécurité des données et gestion de projet en équipe.

Sur les 10 objectifs fonctionnels définis, 7 ont été entièrement atteints, 1 partiellement atteint et 2 reportés faute de temps. Le taux d'atteinte des fonctionnalités prioritaires (MVP) est de 100 %. Le produit livré est fonctionnel, sécurisé et déployable.

Les principales difficultés rencontrées — coordination d'équipe, déploiement des modèles IA, gestion du temps — ont toutes été surmontées et ont généré des apprentissages durables. La décision de documenter systématiquement (API, données, installation, risques) en parallèle du développement a été la meilleure décision organisationnelle du projet.

Ce bilan est rédigé honnêtement, sans minimiser les difficultés ni exagérer les réussites. Il constitue une base solide pour toute équipe qui reprendrait YANSNET dans une itération future ou dans un contexte professionnel.